1. Bank Payment System

Requirements

- Create an abstract class **Bank** with an abstract method `pay()`.
- Create three concrete classes: **UPI**, **CreditCard** and **NetBanking**.
- Each class should implement *pay()* differently.

Concepts

abc, ABC, @abstractmethod, abstract class, abstract method, concrete class.

Program

```
# Task 1: Bank Payment System
from abc import ABC, abstractmethod


class Bank(ABC):
    @abstractmethod
    def pay(self):
        pass


class UPI(Bank):
    def pay(self):
        print("Payment using UPI")


class CreditCard(Bank):
    def pay(self):
        print("Payment using Credit Card")


class NetBanking(Bank):
    def pay(self):
        print("Payment using Net Banking")


for payment in (UPI(), CreditCard(), NetBanking()):
    payment.pay()
```

Output

```
Payment using UPI
Payment using Credit Card
Payment using Net Banking
```

Explanation

Bank is an abstract class: it only declares that every payment type must have a *pay()* method. UPI, CreditCard and NetBanking are concrete classes because each one implements *pay()* in its own way. The same call *payment.pay()* gives a different result for each object (polymorphism).

2. Vehicle System

Requirements

- Create an abstract class **Vehicle** with an abstract method `start()`.
- Create **Car**, **Bike** and **Bus** classes.
- Each class should provide its own implementation of *start()*.
- Also add a concrete method *stop()* in the Vehicle class.

Concepts

abc, ABC, @abstractmethod, abstract method, concrete method.

Program

```
# Task 2: Vehicle System
from abc import ABC, abstractmethod


class Vehicle(ABC):
    @abstractmethod
    def start(self):
        pass

    def stop(self):                      # concrete method
        print("Vehicle stopped")


class Car(Vehicle):
    def start(self):
        print("Car starts with a key or push button")


class Bike(Vehicle):
    def start(self):
        print("Bike starts with a kick or self start")


class Bus(Vehicle):
    def start(self):
        print("Bus starts with an ignition switch")


for v in (Car(), Bike(), Bus()):
    v.start()
    v.stop()
    print()

# An abstract class cannot be instantiated
try:
    Vehicle()
except TypeError as e:
    print("Error:", e)
```

Output

```
Car starts with a key or push button
Vehicle stopped

Bike starts with a kick or self start
Vehicle stopped

Bus starts with an ignition switch
Vehicle stopped

Error: Can't instantiate abstract class Vehicle without an implementation for abstract method
    'start'
```

Explanation

start() is abstract, so Car, Bike and Bus must implement it. *stop()* is a concrete method with a ready implementation in Vehicle, so all child classes inherit it without rewriting it. The last line of the output shows that an abstract class cannot be instantiated: Python raises a **TypeError**.

3. Employee Salary

Requirements

- Create an abstract class **Employee** with an abstract method `calculate_salary()`.
- Create **FullTimeEmployee**, **PartTimeEmployee** and **Intern** classes.
- Each class should calculate the salary differently.
- Also create a concrete method *display()* that prints the employee name.

Concepts

abc, ABC, @abstractmethod, abstract method, concrete method, constructor.

Program

```
# Task 3: Employee Salary
from abc import ABC, abstractmethod


class Employee(ABC):
    def __init__(self, name):
        self.name = name

    @abstractmethod
    def calculate_salary(self):
        pass

    def display(self):                  # concrete method
        print("Employee Name:", self.name)


class FullTimeEmployee(Employee):
    def __init__(self, name, monthly_salary):
        super().__init__(name)
        self.monthly_salary = monthly_salary

    def calculate_salary(self):
        return self.monthly_salary


class PartTimeEmployee(Employee):
    def __init__(self, name, hours_worked, rate_per_hour):
        super().__init__(name)
        self.hours_worked = hours_worked
        self.rate_per_hour = rate_per_hour

    def calculate_salary(self):
        return self.hours_worked * self.rate_per_hour


class Intern(Employee):
    def __init__(self, name, stipend):
        super().__init__(name)
        self.stipend = stipend

    def calculate_salary(self):
        return self.stipend


employees = [
    FullTimeEmployee("Ravi", 50000),
    PartTimeEmployee("Sneha", 80, 300),
    Intern("Alekhya", 15000),
]

for emp in employees:
    emp.display()
    print("Salary:", emp.calculate_salary())
    print()
```

Output

```
Employee Name: Ravi
Salary: 50000

Employee Name: Sneha
Salary: 24000

Employee Name: Alekhya
Salary: 15000
```

Explanation

Each class calculates salary in its own way: FullTimeEmployee returns a fixed monthly salary, PartTimeEmployee multiplies hours worked by the hourly rate (80 x 300 = 24000), and Intern returns a stipend. The concrete method *display()* is written once in Employee and prints the name for every type of employee.

4. Shape Area

Requirements

- Create an abstract class **Shape** with an abstract method `area()`.
- Create **Circle**, **Rectangle** and **Triangle** classes.
- Implement *area()* differently in each class.
- Also add a concrete method *display()* in the Shape class.

Concepts

abc, ABC, @abstractmethod, abstract method, concrete method.

Program

```
# Task 4: Shape Area
from abc import ABC, abstractmethod


class Shape(ABC):
    @abstractmethod
    def area(self):
        pass

    def display(self):                  # concrete method
        print(f"Area of {self.__class__.__name__}: {self.area()}")


class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return round(3.14159 * self.radius ** 2, 2)


class Rectangle(Shape):
    def __init__(self, length, width):
        self.length = length
        self.width = width

    def area(self):
        return self.length * self.width


class Triangle(Shape):
    def __init__(self, base, height):
        self.base = base
        self.height = height

    def area(self):
        return 0.5 * self.base * self.height


for shape in (Circle(5), Rectangle(4, 6), Triangle(10, 8)):
    shape.display()
```

Output

```
Area of Circle: 78.54
Area of Rectangle: 24
Area of Triangle: 40.0
```

Explanation

Circle uses πr^2 (3.14159 x 5 x 5 = 78.54), Rectangle uses length x width, and Triangle uses 1/2 x base x height. The concrete method *display()* in Shape calls *self.area()*, so it automatically uses the correct formula of whichever child class the object belongs to.

5. Online Food Ordering

Requirements

- Create an abstract class **FoodOrder** with an abstract method `place_order()`.
- Create **PizzaOrder**, **BurgerOrder** and **BiryaniOrder** classes.
- Each class should implement *place_order()* differently.
- Also create a concrete method *order_status()* in the parent class.
- The concrete method should print: *Order is being processed*

Concepts

abc, ABC, @abstractmethod, abstract class, concrete method.

Program

```
# Task 5: Online Food Ordering
from abc import ABC, abstractmethod


class FoodOrder(ABC):
    @abstractmethod
    def place_order(self):
        pass

    def order_status(self):              # concrete method
        print("Order is being processed")


class PizzaOrder(FoodOrder):
    def place_order(self):
        print("Pizza order placed")


class BurgerOrder(FoodOrder):
    def place_order(self):
        print("Burger order placed")


class BiryaniOrder(FoodOrder):
    def place_order(self):
        print("Biryani order placed")


for order in (PizzaOrder(), BurgerOrder(), BiryaniOrder()):
    order.place_order()
    order.order_status()
    print()
```

Output

```
Pizza order placed
Order is being processed

Burger order placed
Order is being processed

Biryani order placed
Order is being processed
```

Explanation

place_order() is abstract, so each food order class gives its own message. *order_status()* is a concrete method defined once in FoodOrder and prints "Order is being processed" for every child class.